구동안내서 — 무사히(현장 위험성평가 AI · 작업 전 위험성평가)

2026 공공 AI 대전환 챌린지 세션1 · AI 정부 실험실 챔피언십(자유주제) 출품작 · 세션2 양식2 항목 순서를 따라 작성(세션1 제출 양식이 다르면 항목만 옮겨 쓰면 됨)

접속 주소: 공개 시연판 https://safety-risk.onrender.com (모의자료 전용, 설치 없이 확인) · 설치 실행 시 http://localhost:8105

1. 실행 환경

- 운영체제: Windows 10/11 (macOS·Linux도 `run.sh`로 실행)
- 브라우저: Chrome 또는 Edge 최신판 권장(개발 중 Chromium으로 화면 시험). 앱은 이 PC 안에서만 열립니다(`http://localhost:8105`).
- Windows 바로 실행판(`safety-risk_윈도우_바로실행.zip`)은 Python 설치와 인터넷이 필요 없습니다(Python 3.11과 라이브러리를 `runtime\` 폴더에 함께 넣음). 소스판으로 실행할 때만 Python 3.10 이상이 필요합니다(설치 첫 화면에서 "Add python.exe to PATH" 체크).
- 권장 사양: 모델 없이(규칙 기반) 쓰면 일반 사무용 PC로 충분합니다. 내 PC에서 언어모델(Ollama 등)을 함께 돌리면 모델 크기만큼 메모리와 디스크가 더 필요합니다. 공통기반 등 외부 모델 서버에 연결하면 PC 사양 영향이 없습니다.
- 데이터베이스·별도 서버 설치 없음. 처음 실행 때 라이브러리 설치에 인터넷이 필요하고, 이후에는 인터넷 없이 동작합니다(인터넷이 막힌 PC는 4-3 참고).

2. 설치·실행 절차

1. 바로 실행판(권장): `safety-risk_윈도우_바로실행.zip`의 압축을 풀고 `run.bat`을 더블클릭합니다. 설치 단계 없이 바로 시작합니다.
2. 소스판: 저장소를 내려받아 압축을 풀고(경로에 한글이 있어도 됨) `run.bat`을 더블클릭합니다. 처음 한 번은 실행 환경(.venv)을 만들고 라이브러리를 인터넷으로 설치한 뒤 시작합니다.
3. 브라우저에서 http://localhost:8105 화면이 자동으로 열립니다. 끝낼 때는 검은 실행 창을 닫습니다.
4. 동작 확인(선택): `python tests/smoke_test.py` — 주요 기능을 자동으로 호출해 통과 여부를 보여 줍니다.

3. LLM 연결 설정

- 모델이 없어도 모든 기능이 규칙으로 동작합니다. 모델은 낱말로 못 찾은 위험요인 제안, 현장 사진 판독, 문장 쓰기(물어보기 답·위험요인 해설·종합 의견·TBM 문안·오늘 작업 브리핑·문단 다듬기)에 씁니다. 모델 문장에 근거 사실에 없는 숫자나 위험요인 코드가 있으면 버리고 규칙 문장을 씁니다. 글 제안은 목록에 있는 코드이고 근거 문구가 작업 설명에 그대로 있을 때만, 사진 제안은 목록에 있는 코드만 받아 '현장 확인 필요'로 둡니다. 사진 판독은 사진을 받는 모델에서만 동작합니다.
- 바꾸는 곳: 화면 오른쪽 위 모델 배지를 눌러 연결 방식·주소·모델 이름을 고르고 "저장"·"연결 확인"을 누릅니다. 설정은 `config.json`에 저장됩니다(예시: `config.example.json`). 환경변수 `LLM_PROVIDER`, `LLM_BASE_URL`, `LLM_MODEL`로도 바꿀 수 있습니다. 코드 수정은 필요 없습니다.
- API 키 입력 위치: 키는 화면이나 파일에 적지 않고 환경변수로만 읽습니다(기본 이름 `LLM_API_KEY`, 화면에서 이름 변경 가능). Windows에서는 시작 메뉴 → "계정의 환경 변수 편집" → 새로 만들기로 넣고 앱을 다시 실행합니다.
- 연결 방식: OpenAI 호환 API(행정안전부 AI 공통기반, vLLM, Ollama `/v1` 등), Ollama 기본 API.
- 업스테이지 솔라 예: 연결 방식 `openai`, 주소 `https://api.upstage.ai/v1`, 모델 `solar-pro4`, 키는 환경변수 `LLM_API_KEY`. 공개 시연판은 이 값으로 연결하며, 화면에서 모델 설정을 바꿀 수 없습니다(키 유출 방지). 모델 호출은 1분에 40번으로 묶습니다.
- 지원 모델 계열: 젬마(Gemma)·라마(Llama)·GPT-OSS·엑사원(EXAONE)·하이퍼클로바X·솔라(Solar). 설정 화면의 "모델 목록 불러오기"로 서버에 실제 등록된 모델 이름을 받아 고릅니다. 계열별 응답 차이(system 역할 미지원, 생각 과정 태그, 추론형 모델의 빈 답)는 자동으로 처리합니다.

- 2종 이상 모델 검증: Ollama 설치 후 `모델검증.bat`을 실행하면 젬마·엑사원으로 위험요인 제안·사진 판독을 돌려 `verify_report.md`를 만듭니다. 다른 주소의 모델은 `python tools/verify_models.py --base-url <주소> --models <모델1> <모델2>`.

3-1. 공공데이터 연동(선택: 기상청 날씨·긴급재난문자·법령 원문)

- 키가 없어도 동작합니다. 키가 없거나 연결이 안 되면 예시 자료로 같은 흐름을 보여 주고, 화면에 "예시 자료"라고 표시합니다.
- 기상청(공공데이터포털 data.go.kr에서 「기상청_단기예보 조회서비스」, 「기상청_기상특보 조회서비스」 활용신청 → 일반 인증키(Decoding)): 환경변수 `DATA_GO_KR_KEY`. 기상청 API허브(apihub.kma.go.kr)에서 「동네예보 조회 > 초단기실황조회」와 기상특보 조회를 신청했다면 그 인증키를 `KMA_APIHUB_KEY`로 넣어도 됩니다(한돌과 같은 키). 지금 기온·습도로 기상청 여름철 체감온도 식을 써서 체감온도를 계산하고, 발표 중인 특보를 가져옵니다.
- 긴급재난문자(재난안전데이터공유플랫폼 safetydata.go.kr에서 「긴급재난문자」 활용신청): 환경변수 `SAFETYDATA_KEY`.
- 법령 원문(법제처 국가법령정보 공동활용 open.law.go.kr에서 Open API 활용신청 → API 인증값(OC)): 환경변수 `LAW_OC`. 평가 결과의 위험요인마다 "법령 원문"을 누르면 관련 기준(예: 산업안전보건기준에 관한 규칙, 추락 위험 방지)에 맞는 조문 번호·본문·시행일을 법제처에서 받아 보여 주고, 평가표에 "관련 법령 조문"으로 넣습니다. 인증값이 없으면 조문을 지어내지 않고 국가법령정보센터 링크만 보여 줍니다.
- 넣는 곳: PC는 앱 폴더의 `keys.example.bat`을 `keys.bat`으로 바꾸고 `=` 뒤에 키를 붙여넣습니다(`run.bat`이 읽음, 저장소에는 올리지 않음). Render 공개 시연판은 서비스 → Environment에 같은 이름으로 넣습니다.
- 기관 위치는 환경변수 `DISASTER_REGION`(기본 "서울특별시 도봉구"), `WEATHER_NX`·`WEATHER_NY`(기상청 격자, 기본 61·129), `WEATHER_STN`(특보 지점, 기본 109)으로 바꿉니다.

4. 모의데이터와 자료 위치

4-1. 모의데이터

- `sample_data/`는 가상의 공공기관(시설관리) 안전보건 자료입니다: 위험요인 목록 12종, 안전보건관리규정(위험성 판단 기준·수시평가·TBM·작업별 수칙), 정기 평가 대상 작업 9개, 아차사고 4건, 사진 판독 시연용 그림 1장. 실제 기관·인물·사건과 관계없고 민감정보가 없습니다. `tools/samples_safety.py`로 다시 만들 수 있습니다.
- 위험요인 목록의 '관련 기준'은 법령 이름과 주제까지만 적었습니다. 세부 조문은 국가법령정보센터에서 확인해야 합니다.

4-2. 원본 보호와 산출물 위치

- 기관 자료는 작업 폴더(`data/kb`)에 사본으로 두고 읽기만 합니다.
- 평가·아차사고·사진·작업 기록은 `data/` 폴더에만 저장합니다(설정 `data_dir` 또는 환경변수 `APP_DATA_DIR`로 변경). 평가표·TBM 기록지는 화면에서 내려받습니다.
- 화면에 적은 문장 속 주민등록번호·계좌·카드·여권 번호 같은 개인정보는 처리·저장·모델 전달 전에 ○○○로 가리고, 가렸다고 화면에 알립니다.

4-3. 인터넷이 막힌 PC

- 인터넷이 되는 PC에서 같은 버전의 Python으로 `python -m pip download -r requirements.txt -d wheels`를 실행해 만든 `wheels` 폴더를 앱 폴더에 함께 넣으면, `run.bat`이 인터넷 없이 그 파일로 설치합니다.

5. 기능 사용 순서(심사용)

1. 실행하면 모의 기관 자료가 작업 폴더에 자동으로 복사되고 현황이 열립니다.
2. 새 위험성평가에서 "예시 채우기" → "위험성평가 만들기"를 누릅니다. 위험요인 4개와 가능성·중대성 판단 이유(높이·1인 작업·젖은 바닥·최근 아차사고), "작업 전 조치 필요" 판정이 나옵니다.
3. 위험성 6 이상 항목의 감소대책을 체크하면 "작업 시작 가능"으로 바뀝니다. 가능성·중대성을 바꾸면 '담당자 조정'으로 기록됩니다.
4. "작업 전 회의(TBM)"에서 낭독 문안을 보고 서명 기록지를, "한글(HWPX)"로 위험성평가표를 내려받습니다.

5. 정기 평가 대상 작업에서 '수영장 염소 약품 투입'을 골라 평가한 뒤, 아차사고에서 "예시 채우기" → "등록"을 누르면 그 평가에 '수시평가 필요'가 표시됩니다.
6. 지하 저수조 청소처럼 밀폐공간 작업을 평가하면 작업허가서 안내가, 실외 작업에 체감온도가 없으면 질문이 나옵니다.
7. 새 위험성평가에서 시설 "본부 청사", 작업 "옥상 냉각탑 점검·청소", 설명 "옥상 냉각탑 위 높이 3m 사다리 작업, 2명이 작업"을 적고 "기상청 날씨"를 누른 뒤 평가를 만들면, 체감온도가 채워지고 강풍주의보(예시 자료) 때문에 "작업 전 조치 필요"로 막힙니다. 현황의 "재난문자 확인" → "수시평가 필요 표시"를 누르면 실외 작업 평가에만 표시되고, TBM 문안에 날씨·작업 중지 사유·재난문자가 들어갑니다.
8. 현황 맨 위 오늘 작업 브리핑에서 평가마다 기상특보·체감온도·재난문자·아차사고·미완료 조치를 엮은 판정(옥상 냉각탑 = 작업 중지)을 보고 "AI 브리핑 쓰기"를 누릅니다.
9. 물어보기에서 "사다리 작업 안전 기준은?"을 물은 뒤 "그럼 2m 넘으면?"을 이어 묻습니다. 앞 질문의 위험요인을 이어받았다는 표시와 근거(규정 조문·위험요인 목록)를 확인합니다.
10. 평가를 열고 "미리보기·고쳐서 내려받기"를 누르면 왼쪽(초안 입력 칸)과 오른쪽(미리보기)으로 나뉩니다. "위험요인마다 AI 해설 넣기", "AI로 종합 의견 쓰기", "+ 사진"을 눌러 본 뒤 한글로 내려받습니다. "작업 전 회의(TBM)" 창의 "AI로 말하기 쉽게 다시 쓰기"도 눌러 봅니다.
11. 기준·자료에서 규정 조문에서 읽은 판단 기준, 위험성 행렬, 위험요인 목록을 확인합니다.
12. 서비스 구조·검증 탭에서 처리 단계, 틀린 답을 막는 장치, 정확도 성적표(틀린 문항 포함)와 측정 이력을 확인합니다. 같은 점수는 `python tools/eval.py`로 누구나 다시 낼 수 있습니다(모델·인터넷 불필요).

6. 알려진 제약 사항

- 위험요인은 목록의 낱말로 찾으므로 목록에 없는 표현은 놓칠 수 있습니다. 빠진 위험은 화면에서 추가하고, 목록 엑셀에 낱말을 보태면 다음부터 찾습니다.
- 가능성·중대성 조정 규칙(1인 작업 +1, 아차사고 +1 등)은 출발점이며, 담당자가 현장에서 확인해 고친 값이 평가표에 들어갑니다. 개선 후 위험성은 '감소대책을 모두 하면 가능성이 1'이라는 가정으로 계산합니다.
- 아차사고는 작업 이름이 겹치고 같은 시설일 때 같은 작업으로 봅니다. 반영 기간은 최근 12개월(기본값)입니다.
- 사진 판독은 가짜 모델 서버로 연결 경로만 시험했고, 실제 비전 모델의 정확도는 시험하지 못했습니다. 시연용 사진은 그림입니다.
- 화면은 이 PC에서만 열리며(127.0.0.1), 다른 웹사이트에서 보낸 요청은 거부합니다. 로그인·권한 관리는 없습니다.
- 공공데이터 연동은 이 개발 환경에서 기상청·재난안전데이터공유플랫폼 서버에 접속할 수 없어, 응답 모양을 흉내 낸 자료로 읽기 처리만 시험했습니다. 실제 키로 처음 연결할 때 화면의 출처 표시가 "예시 자료"에서 기관 이름으로 바뀌는지 확인해야 합니다. 특보 해제 여부는 최근 발표 제목의 '해제' 글자로 판단합니다.
- 재배포 제한 라이브러리: 없음. 사용 라이브러리는 python-hwpx(Apache-2.0), lxml·pypdf(BSD-3-Clause), openpyxl·python-docx(MIT), olefile(BSD)입니다. 화면은 외부 라이브러리 없이 직접 작성했습니다.
- 정확도 성적표 문항은 개발자가 만든 것이라 블라인드 시험이 아닙니다. 그래서 앱을 고칠 때 보지 않은 묶음(B)을 따로 잽니다: 위험요인 찾기 B 27/30 (A 묶음 30/30). 실제 언어모델로 잰 결과는 `모델검증.bat` 실행 후 화면 머리에 표시되며, 제출 시점 측정값은 `검증결과/`에 있습니다.
- 생성형 AI 문장은 이 개발 환경에서 업스테이지 서버에 접속할 수 없어 같은 형식의 가짜 모델 서버로 시험했습니다(자동 점검 50개 항목). 실제 솔라 답은 공개 시연판에서 확인해야 합니다.
- 공개 시연판(환경변수 `PUBLIC_DEMO=1`)은 키 유출을 막으려고 화면의 모델 설정 변경을 막고(모델은 서버 환경변수로 설정) 한 번에 40MB까지만 받으며, 무료 호스팅이라 재시작하면 넣은 자료가 지워집니다. 실제 업무자료는 기관 PC 설치판에서만 쓰세요.

7. 개발 경위 요약

- 개발: 공공기관 【부서·성명】, 2026년 10월.
- 방식: 담당자가 현장 문제와 요구사항을 정하고, AI 코딩 도구로 코드를 작성·수정하는 방식(바이브코딩)으로 개발했습니다. 기능마다 자동 점검(`tests/smoke_test.py`)과 가짜 모델 서버를 이용한 모델 연결 시험

(`tools/verify_models.py --mock`)을 거쳤습니다.

- 설계 원칙: ① 모든 판단에 이유와 원문 근거 표시, ② 판정은 규정 기준과 규칙으로 하고 모델은 보조(대조 통과한 제안만 사용, 사진은 담당자 확인 후 반영), ③ 원본 파일은 읽기만 함, ④ 외부 웹 프레임워크·데이터베이스 없이 파이썬 기본 기능으로 구성해 기관 이관이 쉬움.
- 참고한 공개 개발물: 없음(위 라이브러리 외 전부 직접 작성).

8. 문의

공공기관 【부서·성명·연락처】